

Search-Space Reduction in Neural Architecture Search: Does It Help or Just Reshuffle the Budget?

Sergei Kurashkin

*AI Technology Scientific and Education Center
Bauman Moscow State Technical University
Department of Applied Artificial Intelligence
RUDN University
Moscow, Russia
kurashkin@bmstu.ru*

Vladimir Nelyub

*Bauman Moscow State
Technical University
Moscow, Russia
vladimir.nelub@emtc.ru*

Aleksei Borodulin

*Bauman Moscow State
Technical University
Moscow, Russia
alexey.borodulin@emtc.ru*

Vadim Tynchenko

*Bauman Moscow State
Technical University
Moscow, Russia
tynchenko@bmstu.ru*

Abstract—Search-space reduction is routinely credited with accelerating neural architecture search (NAS): a proxy score or an operation rule prunes the space before the search. Published evaluations rarely compare the reduced space with a random subspace of the same size at an equal query budget, so it remains unclear whether the selection rule or the smaller size drives the reported gains. We introduce an equal-budget, size-matched evaluation protocol and audit four published reductions (operation pruning and top-50% selection by synflow, NASWOT and parameter count) under four search algorithms (random search, regularised evolution, REINFORCE and local search) on the three datasets of the NAS-Bench-201 topology space, with 200 seeds per condition. On this benchmark, of 48 dataset–algorithm–reduction cells at the final budget of 1,000 queries, 27 show no Holm-significant difference from the matched control, 12 favour the reduction and 9 the control. Every final effect is at most 0.373 percentage points, smaller than one across-seed standard deviation, and half of the significant gains belong to the weakest searcher. The selection rules do help early, beating their controls by 0.71–1.26 points at 10 queries, but the advantage fades with budget and in several cells turns into a significant loss. Full-space searches already spend 89–91% of their best trajectories inside the regions that score-based reductions keep, so the reductions mostly reshuffle where the budget is spent rather than improve where it ends. Size-matched random controls at equal budgets should become a standard baseline for search-space reduction; code and frozen subspace definitions are released for exact reproduction.

Index Terms—neural architecture search, search-space reduction, automated machine learning, evolutionary computation, benchmarking, statistical evaluation

I. INTRODUCTION

Neural architecture search automates the design of network topologies, and its cost is dominated by the number of candidate architectures that must be trained and evaluated [1], [2]. A widely adopted route to cutting this cost is to shrink the search space before or during the search: remove operations believed to be unhelpful, keep only the architectures ranked highly by a zero-cost proxy, or restrict the space by a structural criterion such as parameter count [3]–[8]. The accompanying claim is that searching the reduced space reaches better architectures sooner, and the claim is usually supported by comparing a search in the reduced space against the same search in the full space.

That comparison conflates two different explanations. A reduced space can help because the selection rule concentrates quality, or simply because any smaller space is easier to cover at a fixed number of queries. Separating the two requires a control that published evaluations rarely include: a random subspace of exactly the same size, searched under exactly the same query budget. Prior critiques of NAS evaluation have exposed weak baselines and under-reported variance when comparing search *algorithms* [9]–[12]; to our knowledge, no study has applied the same discipline to the comparison of search *spaces*, where the confound is subspace size rather than tuning effort. Seed variance compounds the problem: on tabular NAS benchmarks the spread between seeds is often larger than the differences under study (Table II), so evaluations based on a handful of runs cannot resolve them.

We close this gap with an equal-budget, size-matched evaluation protocol (Algorithm 1, Fig. 1). Four published reduction heuristics are instantiated on the NAS-Bench-201 topology space [13], [14]: operation pruning that removes the `none` operation, and top-50% selection by synflow, by NASWOT and by parameter count. Each reduced space is paired with a random subspace of the same cardinality, re-sampled per seed. Four algorithms search every space: random search, regularised evolution [15], REINFORCE [16] and first-improvement local search [17]. They share a budget of 1,000 noisy validation queries, and each condition runs 200 seeds, compared in pairs on the final test accuracy.

Contributions, scope and limitations. The paper contributes (i) an equal-budget, size-matched evaluation protocol for search-space reductions. It carries the benchmarking discipline of evolutionary computation over to search-space reduction (budgets counted in evaluations with every proposal charged, independent runs, paired non-parametric tests, anytime reporting) and adds the null model this practice lacks, a random subspace of the same size, together with a leakage-safe select-on-validation, report-on-test rule. The paper also contributes (ii) a large-seed audit of four published reduction heuristics across four search algorithms and three datasets on NAS-Bench-201, with 200 seeds per condition and 16,800 searches in total. It contributes (iii) evidence, on this bench-

mark, that at the final budget most of the audited reductions perform no better than size-matched random subspaces. Their early advantage comes from the selection rule rather than from size and fades once the budget grows past a small fraction of the space, so the reductions largely reshuffle where the budget is spent. Finally, it contributes (iv) released code and frozen subspace definitions for exact reproduction. The findings are empirical statements about four reductions on one benchmark for single-objective image classification; the protocol itself applies to any search space in which membership can be tested and sampled. We do not claim that reductions are useless in open-world spaces, only that their standard evidence does not separate rule from size. For the evolutionary data mining and machine learning audience specifically, the protocol gives evolutionary NAS a fair, variance-aware yardstick: regularised evolution is evaluated under the same budget discipline as every other algorithm, and the reduction heuristics, not the evolutionary search itself, are what the audit interrogates.

II. RELATED WORK

Search-space design and reduction. A first line of work treats the search space itself as the object to optimise. Angle-based shrinking [3] and evolved search spaces [4] prune candidates during the search; curriculum approaches grow the space progressively [6]; design-space analysis distils large spaces into small well-behaved families [5]; and learned space partitions steer sampling towards promising regions [18]. Zero-cost proxies such as synflow [19], NASWOT [7] and their aggregations [8] offer a cheap ranking signal that is frequently proposed as a pre-filter. In this literature the reduced space is usually evaluated against the full space or against the unfiltered search, at matched wall-clock or query cost. Such a comparison cannot separate the benefit of the selection rule from the benefit of a smaller space; a random subspace of the same size, searched at the same budget, is the control that does.

Critiques of NAS evaluation. A second line established that NAS comparisons are fragile: random search with early stopping matches or beats published algorithms [9], the search phase contributes less than the training pipeline [10], weight-sharing rankings correlate poorly with true rankings [11], and community best practices call for matched budgets, repeated seeds and released code [12]. Predictor-focused audits extended this discipline to performance estimators [20], [21]. These works compare *algorithms* or *estimators*; the present study applies the same discipline to *spaces*, where the natural null model is not random search but a random subspace of matched cardinality, searched at the same budget.

Tabular benchmarks. Exhaustively evaluated benchmarks, NAS-Bench-101 [22], NAS-Bench-201 and its NATS-Bench extension [13], [14], and the derived suites with precomputed proxy scores [23], [24], make seed-rich, statistically grounded comparisons affordable: a query returns a recorded training trial instead of a fresh training run. They are the natural substrate for a protocol whose cost grows with seeds times budget, and we use them accordingly.

Evolutionary NAS and benchmarking in evolutionary computation. Evolutionary computation is one of the founding search paradigms of NAS [15], [25] and remains central to automated machine learning within the evolutionary data mining community [26], [27], where population-based methods are also used to optimise network structure directly [28]. Benchmarking practice in evolutionary computation supplies most of the discipline used below: budgets counted in function evaluations [29], repeated evaluation of the same solution as a standard response to noisy fitness [30], and many independent runs compared with paired non-parametric tests and multiple-comparison corrections [31]. What this practice does not provide is a null model for a restriction of the representation itself. A search-space reduction is such a restriction, and the size-matched random subspace used here is its null model: a random constraint of the same volume, searched under the same evaluation budget. The protocol treats regularised evolution as a first-class participant and asks whether the reductions, not the evolution, earn their keep.

III. EQUAL-BUDGET EVALUATION PROTOCOL

A. Benchmark and Datasets

All experiments run on the topology search space of NAS-Bench-201 [13] through its NATS-Bench interface [14]: cells are directed acyclic graphs over four nodes whose six edges each carry one of five operations, giving $5^6 = 15,625$ architectures, each exhaustively trained on CIFAR-10 (with the benchmark’s validation protocol, denoted `cifar10-valid`), CIFAR-100 and ImageNet16-120 [32] under the 200-epoch schedule. The benchmark archive is pinned by its SHA-256 digest (prefix `580fd8f3`; full value in the repository). Every run therefore resolves queries against bit-identical data.¹ A query returns the validation accuracy of one recorded training trial, sampled among the stored trials of the queried architecture; queries are therefore noisy, as they are in deployment, and the noise is reproducible under the per-unit seeding described below. Final performance is reported as the mean test accuracy of the architecture with the best observed validation accuracy: selection sees validation data only, and test accuracy is touched exactly once per search (Fig. 1).

B. Search-Space Reductions

Table I lists the seven spaces; the four informed reductions are chosen to cover the three published families of reduction rule: an operation-level pruning rule, two zero-cost proxy filters, and a structural criterion. The full space is the untouched benchmark. The operation-pruning reduction NO-NONE removes the `none` operation from the candidate set, keeping the $4^6 = 4,096$ cells that use only the remaining four operations. Three score-based reductions keep the top 50% of architectures (7,813 cells) ranked by synflow [19], by NASWOT [7], or by parameter count, with the proxy scores taken

¹Code, frozen space memberships, all per-run and aggregated results, and an extended version of this paper with additional figures and the full per-checkpoint table: <https://github.com/digger32/Search-Space-Reduction-in-NAS>.

TABLE I

SEARCH SPACES. INFORMED REDUCTIONS ARE FROZEN PER DATASET WITH PUBLISHED MEMBERSHIP HASHES; RANDOM CONTROLS ARE RESAMPLED PER SEED AT MATCHED CARDINALITY.

Space	Construction rule	Size
FULL	complete NAS-Bench-201 topology space	15,625
NO-NONE	drop the <code>none</code> operation	4,096
SYNFLOW-50	top 50% by synflow score	7,813
NASWOT-50	top 50% by NASWOT score	7,813
PARAM-50	top 50% by parameter count	7,813
RAND-4096	uniform random, per-seed	4,096
RAND-7813	uniform random, per-seed	7,813

from the precomputed NAS-Bench-Suite-Zero release [24]. Each informed reduction is frozen before any search: the membership list is computed once per dataset, hashed, and the hashes are published with the code, so no result can silently move an architecture across the boundary. The decisive controls are the two size-matched random subspaces, RAND-4096 and RAND-7813, drawn uniformly without replacement to the cardinality of the informed reductions and resampled independently for every seed; a reduction whose only advantage is size cannot beat its matched control in expectation.

C. Search Algorithms

Four algorithms span the range from memoryless to strongly exploitative. Random search (RS) draws architectures uniformly from the space. Regularised evolution (RE) [15] maintains a population of 10 with tournament size 3 and single-edge mutation, discarding the oldest individual. REINFORCE [16] learns an edge-factorised categorical policy with learning rate 0.01 and an exponential moving-average baseline with decay 0.9. Local search (LS) [17] performs first-improvement hill climbing over single-edge neighbourhoods with uniform restarts on convergence. Hyperparameters are frozen at these values across every space, dataset and budget: no condition receives tuning that another does not, so there is no hyperparameter optimisation anywhere in the design. Proposals are closed within the active space by bounded rejection (at most 100 resampled attempts per proposal) with a uniform in-space fallback, so an algorithm designed for the full space cannot step outside a reduction. The cost of this projection is recorded and is modest: random search and local search sample inside the space by construction and never reject, whereas regularised evolution and REINFORCE discard 0.37–1.44 out-of-space attempts per accepted proposal in the informed reductions. The same algorithms reject more often in the size-matched random controls (0.92–3.73 attempts per proposal), because a scattered random subspace is less closed under single-edge mutation than an informed one; the control is therefore not favoured by the projection. At the observed acceptance rates the 100-attempt cap is reached with probability below 10^{-10} , so the fallback never distorts a search.

D. Budget Semantics

One query is one recorded training trial’s validation accuracy, and every proposal consumes budget, including proposals

Algorithm 1 Equal-budget search under a reduction

Require: space S (membership set), algorithm $\mathcal{A}$, budget B , seed s

- 1: initialise $\mathcal{A}$ with rng s ; best $\leftarrow -\infty$
- 2: **for** $t = 1$ **to** B **do**
- 3: $a_t \leftarrow \text{propose}(\mathcal{A})$ restricted to S (bounded rejection, uniform fallback)
- 4: $v_t \leftarrow$ validation accuracy of one recorded trial of a_t (noisy)
- 5: update $\mathcal{A}$ with (a_t, v_t) ; if $v_t > \text{best}$: best $\leftarrow v_t$, $a^* \leftarrow a_t$
- 6: **end for**
- 7: **return** mean test accuracy of a^* ; anytime trace at checkpoints

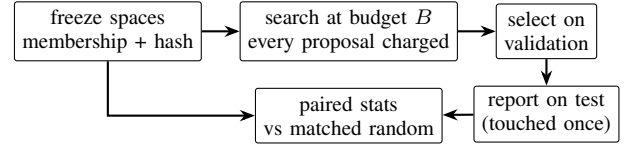


Fig. 1. Leakage-safe evaluation flow. Spaces are frozen and hashed before any search; selection uses validation accuracy only; test accuracy enters once, at reporting; statistics pair each seed with its size-matched random control.

of architectures seen earlier in the same search. This rule matches the deployed cost model, in which a re-evaluation is a real training run; it makes re-evaluation informative, since trials are noisy and a second draw refines the estimate; and it removes the pathology of converged policies proposing one architecture forever at zero cost. The budget is $B = 1,000$ queries with anytime checkpoints at $\{10, 25, 50, 100, 200, 500, 1,000\}$, and each condition runs seeds 0–199. Per-unit determinism comes from seeding the benchmark’s trial sampling globally and the algorithm’s proposal stream per seed, which is what allowed the final pass to reproduce the preceding full pass bit-identically.

E. Statistical Protocol

The unit of analysis is one search, and comparisons are paired by seed: seed s under a reduction is compared with seed s under the size-matched random control, on the same dataset with the same algorithm. Within each cell the paired final test accuracies are tested with the Wilcoxon signed-rank test using Pratt’s treatment of zeros [33], [34], which matters here because both branches of a pair frequently select the same architecture (median share of exact ties 0.20 across cells). p -values are Holm-corrected [35] within each dataset–algorithm family of eight comparisons, in which each of the four reductions is tested against its size-matched control and against FULL; this is every test run for one dataset and algorithm, the setting in which a practitioner chooses a reduction. Correcting across all 48 reduction–control comparisons at once is reported as a robustness check in Section IV-A. This design of repeated runs, paired non-parametric tests and Holm correction follows the practice recommended for comparing evolutionary algo-

gorithms [31]. Effect sizes accompany every test: the mean paired difference with a bootstrap 95% confidence interval (10,000 resamples) [36], the Hodges–Lehmann estimator [37], and the counts of positive, zero and negative differences. An omnibus Friedman test [38], [39] compares all seven spaces over the 12 dataset–algorithm blocks. Because it does not reject (Section IV-A), the Nemenyi post-hoc ranking is descriptive only; its critical-difference diagram is in the extended version. Compute is modest by construction: the full grid of 672 units and 16,800 searches costs 33.5 CPU-hours (median 179 s per unit, maximum 260 s) in a pinned environment listed in the repository, and the final pass reruns from scratch with resume disabled.

IV. RESULTS

Each subsection states one claim and is supported by one display item; all numbers come from the gated final run.

A. Most Reductions Do Not Outperform Size-Matched Random Subspaces

Table III reports, for each of the 48 dataset–algorithm–reduction cells, the mean paired difference in final test accuracy between the reduction and its size-matched random control, with Holm-corrected significance. At the full budget of 1,000 queries, 27 of 48 cells show no significant difference, 12 favour the reduction and 9 favour the control. Every effect is small: the largest mean difference in either direction is 0.373 percentage points, and most are below 0.1. Half of the significant gains belong to the weakest searcher: 6 of the 12 cells favouring a reduction are random-search cells, with the largest single effect at `cifar100`, random search under NO-NONE (mean +0.373 pp, Hodges–Lehmann +0.417, sign counts 138/7/55, $p_{\text{Holm}} < 10^{-4}$). The remaining gains are small and dataset-bound: local search gains under all four reductions on `cifar100` (+0.022 to +0.166 pp), regularised evolution once (`cifar100`, NO-NONE, +0.205 pp) and REINFORCE once (ImageNet16-120, NASWOT-50, +0.201 pp). Reductions significantly hurt the stronger searchers on `cifar10-valid`. Local search loses under all four reductions (mean differences −0.052 to −0.071 pp), regularised evolution under NO-NONE, SYNFLOW-50 and PARAM-50, and REINFORCE under NO-NONE. One further loss occurs for local search on ImageNet16-120 under NASWOT-50 (−0.080 pp). The Friedman test over the seven spaces does not separate them globally ($\chi^2 = 10.08$, $p = 0.121$), with mean ranks NASWOT-50 3.12, FULL 3.42, SYNFLOW-50 3.46, PARAM-50 3.67, NO-NONE 4.25, RAND-7813 4.75 and RAND-4096 5.33. The non-significant cells are bounded nulls rather than underpowered comparisons. With 200 paired seeds, the bootstrap 95% interval for the mean difference has a median half-width of 0.075 pp (maximum 0.122 pp) across those 27 cells. No interval endpoint reaches 0.27 pp in either direction, so any effect larger than about a quarter of a percentage point would have been detected. The picture also survives a stricter correction: applying Holm across all 48 reduction–control comparisons at once leaves

31 cells non-significant, 10 favouring the reduction and 7 the control. Statistical significance should not be read as practical relevance here. Every final mean difference is smaller than one across-seed standard deviation of its cell (0.02–0.65 pp, Table II), and the standardised paired effects, the mean difference divided by the standard deviation of the paired differences, are at most 0.49 with a median of 0.16. With 200 paired seeds the tests resolve differences far below what would change a model-selection decision; differences of practical size occur only at the smallest budgets (Section IV-B). The absolute stakes are equally modest (Table II): across all seven spaces and four algorithms the spread of mean final test accuracy is 0.13 pp on `cifar10-valid` (91.04–91.17), 0.64 pp on `cifar100` (72.51–73.16) and 0.40 pp on ImageNet16-120 (46.05–46.45).

B. Early Gains Come from the Selection Rule and Fade with Budget

Figure 2 plots the paired advantage of every reduction over its size-matched control against the budget, averaged over datasets, and Table IV summarises it at every checkpoint. At 10 queries every one of the 16 algorithm–reduction pairs favours the reduction, by 0.71 to 1.26 percentage points. The advantage then decays with budget in all 16 pairs: monotonically for random search, regularised evolution and local search, while REINFORCE falls to near zero by 100–200 queries and only partly recovers. At 1,000 queries the surviving advantages are 0.02 to 0.17 pp. For local search the score-based reductions (SYNFLOW-50, NASWOT-50, PARAM-50) end below zero (−0.02 to −0.04 pp), and REINFORCE under PARAM-50 dips negative in mid-budget (−0.08 pp at 200 queries) before recovering towards zero.

Because each reduction and its control have the same size, the early advantage cannot come from size, and a direct comparison confirms it. For RS a random subspace changes nothing in expectation, and its random subspaces stay within 0.13 pp of FULL at every checkpoint. For the mutation-based RE and LS they are slightly worse than FULL (down to −0.26 and −0.38 pp in mid-budget), because a scattered subspace breaks single-edge moves. The early advantage is therefore the selection rule concentrating quality where a broad search samples first: against FULL, the informed reductions lead by 0.59–1.03 pp at 10 queries, almost all of it rule rather than size. The advantage fades once the budget reaches the high-accuracy tail, which a random subspace of the same size also retains. Table IV locates this boundary by applying the tests of Table III at every checkpoint. The reduction is significantly better than its control in all 48 cells at 10 and 25 queries, in 35 at 100, in 25 at 200 and in 12 at 1,000, where 9 cells have turned significantly worse. The last budget with a significant advantage has a median of 50 queries for REINFORCE, 150 for RE and LS, and 750 for RS, that is, well under 1% of the 15,625 architectures for every searcher except the one that cannot exploit structure.

TABLE II

MEAN FINAL TEST ACCURACY (%) AND, IN BRACKETS, ITS ACROSS-SEED STANDARD DEVIATION OVER 200 SEEDS PER CELL AT $B=1,000$, FOR EVERY DATASET-ALGORITHM-SPACE CELL OF THE GRID. THE SPREAD OF THE MEANS ACROSS SPACES WITHIN A DATASET NEVER EXCEEDS 0.64 PP; THE WEAKEST CELLS ARE RANDOM SEARCH ON THE RANDOM SUBSPACES.

Dataset	Algorithm	FULL	NO-NONE	SYNFLOW-50	NASWOT-50	PARAM-50	RAND-4096	RAND-7813
cifar10-valid	RS	91.05 (0.24)	91.13 (0.17)	91.11 (0.20)	91.07 (0.20)	91.11 (0.20)	91.05 (0.22)	91.04 (0.21)
	RE	91.14 (0.11)	91.12 (0.07)	91.14 (0.08)	91.14 (0.08)	91.13 (0.07)	91.15 (0.17)	91.16 (0.11)
	REINFORCE	91.16 (0.12)	91.12 (0.06)	91.16 (0.10)	91.17 (0.08)	91.17 (0.16)	91.15 (0.16)	91.17 (0.13)
	LS	91.10 (0.04)	91.09 (0.02)	91.10 (0.04)	91.10 (0.04)	91.10 (0.03)	91.16 (0.16)	91.15 (0.11)
cifar100	RS	72.57 (0.54)	72.88 (0.51)	72.67 (0.49)	72.73 (0.49)	72.66 (0.48)	72.51 (0.59)	72.57 (0.61)
	RE	73.14 (0.20)	73.12 (0.09)	73.14 (0.23)	73.16 (0.19)	73.12 (0.23)	72.91 (0.42)	73.06 (0.35)
	REINFORCE	73.13 (0.22)	73.10 (0.04)	73.11 (0.10)	73.11 (0.05)	73.09 (0.37)	72.95 (0.41)	73.05 (0.37)
	LS	73.10 (0.02)	73.09 (0.01)	73.10 (0.01)	73.10 (0.01)	73.10 (0.02)	72.93 (0.40)	73.07 (0.32)
ImageNet16-120	RS	46.07 (0.63)	46.23 (0.55)	46.25 (0.55)	46.19 (0.55)	46.23 (0.55)	46.16 (0.57)	46.05 (0.65)
	RE	46.43 (0.45)	46.45 (0.39)	46.39 (0.34)	46.42 (0.41)	46.41 (0.42)	46.33 (0.44)	46.36 (0.45)
	REINFORCE	46.37 (0.51)	46.36 (0.38)	46.35 (0.42)	46.44 (0.34)	46.34 (0.43)	46.27 (0.52)	46.24 (0.49)
	LS	46.32 (0.31)	46.31 (0.25)	46.37 (0.36)	46.33 (0.29)	46.38 (0.34)	46.35 (0.51)	46.41 (0.43)

TABLE III

MEAN PAIRED DIFFERENCE IN FINAL TEST ACCURACY (PERCENTAGE POINTS, REDUCTION MINUS SIZE-MATCHED RANDOM CONTROL) AT $B=1,000$, 200 SEEDS PER CELL. SIGNIFICANCE AFTER HOLM CORRECTION WITHIN EACH DATASET-ALGORITHM FAMILY OF EIGHT COMPARISONS (SECTION III): $*p < 0.05$, $**p < 0.01$, $***p < 0.001$, $****p < 10^{-4}$. POSITIVE VALUES FAVOUR THE REDUCTION.

Dataset	Algorithm	NO-NONE	SYNFLOW-50	NASWOT-50	PARAM-50
cifar10-valid	RS	+0.078**	+0.068**	+0.031	+0.064*
	RE	-0.033**	-0.020*	-0.019	-0.024**
	REINFORCE	-0.034**	-0.014	-0.002	+0.001
	LS	-0.071****	-0.052****	-0.052****	-0.053****
cifar100	RS	+0.373****	+0.100	+0.153	+0.089
	RE	+0.205****	+0.078	+0.095	+0.061
	REINFORCE	+0.151	+0.065	+0.057	+0.041
	LS	+0.166**	+0.022**	+0.022**	+0.022**
ImageNet16-120	RS	+0.061	+0.199**	+0.143	+0.183*
	RE	+0.125	+0.029	+0.065	+0.055
	REINFORCE	+0.085	+0.104	+0.201****	+0.094
	LS	-0.036	-0.036	-0.080*	-0.028

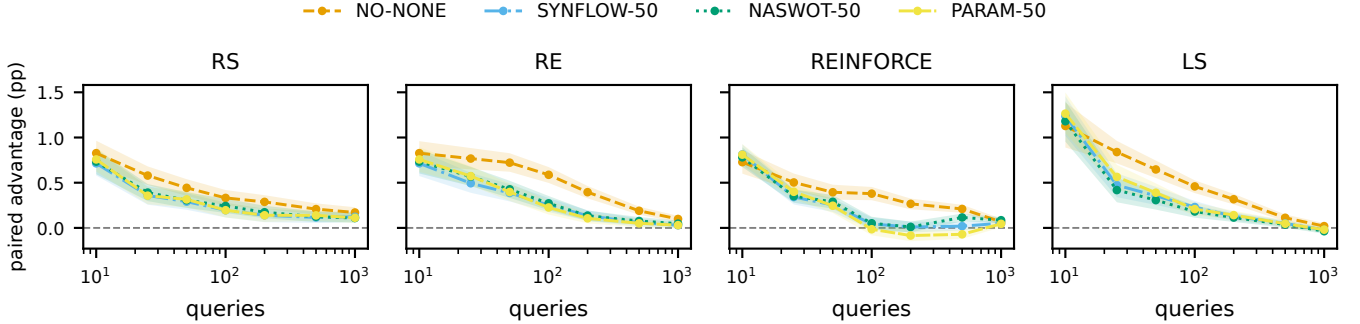


Fig. 2. Paired advantage of each reduction over its size-matched random control against the query budget, per search algorithm, averaged over the three datasets (percentage points; bands: bootstrap 95% intervals). Every reduction starts ahead; the advantage fades with budget and ends near zero, below it for LS under the score-based reductions.

C. Reductions Reshuffle Where the Budget Is Spent

If a reduction merely fences off territory the search would have used anyway, its main effect is a reallocation of queries, not a change in destination. Figure 3 quantifies this on the full-space searches of the same grid: for each informed reduction we measure the fraction of the best-so-far trajectory that already lies inside the region the reduction keeps. For the three score-based reductions this fraction is 0.89–0.91 against

a region size share of 0.50; for NO-NONE it is 0.55–0.60 against a share of 0.26. Unguided search thus concentrates, on its own, in the regions the reductions point to; imposing the fence mostly reshuffles the early budget rather than redirecting the search to territory it would have missed.

TABLE IV

PAIRED ADVANTAGE OF THE REDUCTIONS OVER THEIR SIZE-MATCHED CONTROLS BY BUDGET. MEAN, MIN AND MAX: OVER THE 16 ALGORITHM-REDUCTION PAIRS OF THE DATASET-AVERAGED PAIRED DIFFERENCE (PERCENTAGE POINTS). BETTER AND WORSE: NUMBER OF THE 48 DATASET-ALGORITHM-REDUCTION CELLS IN WHICH THE REDUCTION IS SIGNIFICANTLY BETTER OR WORSE THAN ITS CONTROL (TESTS AND HOLM FAMILY AS IN TABLE III).

Budget B	10	25	50	100	200	500	1,000
Mean	+0.88	+0.50	+0.39	+0.24	+0.15	+0.09	+0.06
Min	+0.71	+0.34	+0.25	−0.01	−0.08	−0.07	−0.04
Max	+1.26	+0.84	+0.72	+0.59	+0.40	+0.21	+0.17
Better	48	48	47	35	25	13	12
Worse	0	0	0	0	0	2	9

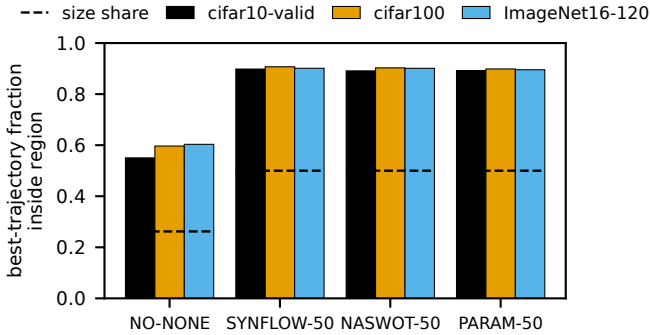


Fig. 3. Fraction of the best-so-far trajectory of full-space searches that falls inside each informed region, against the region’s size share (dashed). Full-space search already spends most of its best trajectory where the reductions point.

D. Effects Depend on the Searcher and Flip Sign Across Algorithms

In Table III the dominant structure is an algorithm axis: moving from random search to local search, the same reduction on the same dataset can flip from a significant gain to a significant loss. On `cifar10-valid`, NO-NONE gives random search +0.078 pp and local search −0.071 pp, both significant. The moderation is also dataset-bound: the same local search that loses under all four reductions on `cifar10-valid` gains under all four on `cifar100`, so a reduction’s sign cannot be predicted from the algorithm alone. The consistency claim that would justify a reduction as a universally safe default is not supported. What is consistent is where the losses fall: all 9 significant losses belong to the searchers that exploit structure (RE, REINFORCE and LS), none to RS, which collects half of the significant gains.

E. Reductions Narrow Evolutionary Search Without Improving Its Result

Table V examines how the reductions interact with the dynamics of the evolutionary searcher, using quantities recorded in every run of RE and a deterministic replay of all 4,200 RE searches that reproduces the reported results bit for bit. Locality: RE discards 0.37–0.66 out-of-space attempts per

TABLE V

SEARCH DYNAMICS OF REGULARISED EVOLUTION PER SPACE, AVERAGED OVER THE THREE DATASETS (200 SEEDS EACH). REJ./PROP.: REJECTED OUT-OF-SPACE ATTEMPTS PER ACCEPTED PROPOSAL. FOUND BY 500: SHARE OF RUNS WHOSE FINAL INCUMBENT IS ALREADY FOUND BY 500 QUERIES. DISTINCT FINAL: DISTINCT FINAL ARCHITECTURES REACHED BY THE 200 RUNS[†]. DIV.: MEAN PAIRWISE HAMMING DISTANCE (0–6) WITHIN THE POPULATION. UNIQUE: DISTINCT ARCHITECTURES EVALUATED. BOTH FROM A REPLAY OF EVERY RUN, VERIFIED BIT FOR BIT AGAINST THE REPORTED RESULTS. [†]RANDOM CONTROLS ARE RESAMPLED PER SEED, SO THEIR COUNT IS NOT COMPARABLE WITH THE FIXED SPACES.

Space	Rej. /prop.	Found by 500	Distinct final	Div. @1,000	Unique @1,000
FULL	0.25	0.54	10.0	2.89	768
NO-NONE	0.66	0.67	5.7	2.77	600
SYNFLOW-50	0.37	0.57	7.3	2.83	716
NASWOT-50	0.42	0.58	8.0	2.85	694
PARAM-50	0.37	0.59	8.3	2.84	722
RAND-4096	3.73	0.68	23.7 [†]	2.42	316
RAND-7813	1.49	0.61	14.3 [†]	2.72	572

accepted proposal in the informed reductions against 1.49–3.73 in the random controls, so an informed subspace is far more closed under single-edge mutation than a random one of the same size. Convergence: the share of runs that have already found their final incumbent after 500 queries rises from 0.54 in FULL to 0.57–0.59 in the score-based reductions and 0.67 in NO-NONE. The random controls settle as early (0.61 and 0.68), so earlier settling follows size, not the rule. End points: the 200 independent runs converge to 10.0 distinct final architectures in FULL and to 5.7–8.3 in the informed reductions. Population diversity is almost untouched: the mean pairwise Hamming distance within the population settles by 100 queries and stays at 2.77–2.85 in the informed reductions against 2.89 in FULL. Exploration breadth is not: in 1,000 queries RE evaluates 600–722 distinct architectures in the informed reductions against 768 in FULL and spends the remaining queries on re-evaluations. A reduction thus narrows where evolutionary search ends without raising the accuracy it ends at (Section IV-A); it does so by concentrating queries on fewer architectures, not by collapsing the population. REINFORCE is the one searcher whose settling slows under the score-based reductions (0.24–0.29 of runs by 500 queries against 0.38 in FULL), in line with its mid-budget dip in Fig. 2.

V. DISCUSSION

The audit returns a consistent explanation across the five claims. A reduction helps a search only while the search is still sampling broadly. The early help is real and comes from the selection rule, since a random subspace of the same size gives no such help; it fades once the budget reaches the high-accuracy tail, which a random subspace of the same size retains as well. At 1,000 queries the advantage is negligible in every cell and occasionally a liability for searchers that exploit structure, because a fence also removes escape routes that local moves would have used. The same asymmetry appears in the projection cost: mutation-based searchers discard between 1.7

and 5.7 times fewer out-of-space attempts inside an informed subspace than inside a random one of equal size (Section III). Informed reductions therefore do buy locality for the search operator, and that locality still fails to convert into final accuracy. Where a selection rule still adds value at the final budget, the gains sit disproportionately with the searcher least able to find structure by itself and on `cifar100`, the dataset with the widest accuracy spread, and even there the largest effect is 0.373 pp. This reading is consistent with the random-search parity results for algorithms [9], [10] and extends them along a new axis: the null model for a reduced *space* must be a random space of the same size, just as the null model for an algorithm is random search at the same budget. It also refines the practical advice attached to zero-cost proxies [8], [24]: their ranking signal, which is genuine, does not automatically convert into an end-to-end search advantage once budget and size are controlled.

When does a reduction help? The per-budget tests of Section IV-B give the answer the full-space comparison cannot. A reduction offers an optimisation advantage beyond a reallocation of queries under three conditions: the budget is a small fraction of the space, the searcher cannot find structure by itself, or the rule concentrates the high-accuracy tail rather than the mean. Here the first condition ends at a median of 50–150 queries for the structure-exploiting searchers, and the second extends it to 750 queries for RS. None of the four audited rules meets the third, which is why their advantage does not persist; a rule that did would keep beating its matched control at large budgets, and the protocol would show it.

Beyond NAS-Bench-201. The conclusions are empirical statements about one benchmark, but the budget analysis indicates how they should transfer. In large search spaces, practical budgets are a far smaller fraction of the space than 1,000 of 15,625, which is the regime in which the rules helped here. Reductions may therefore pay off in such spaces, and the size-matched control matters more there, not less, because only it shows whether the rule or the smaller volume produced a gain. In multi-objective NAS, a reduction by parameter count constrains one objective and truncates the Pareto front, so size matching has to be combined with front-based indicators. In deployed NAS, budgets are measured in training time, and kept and removed regions differ in training cost, which a query budget ignores. These are expectations to be tested rather than findings; the protocol and its code carry over unchanged.

Practical implications for evolutionary data mining and machine learning. For evolutionary AutoML studies the protocol is directly reusable as a default harness: freeze the space memberships with published hashes, charge every proposal to the budget, pair seeds against a size-matched random control, and report effect sizes with corrected significance rather than best-of- k curves. These are the habits of evolutionary benchmarking, applied to a question it has not asked of its own representations: whether a restriction of the search space earns its place against a random restriction of the same size. Regularised evolution itself emerges intact from the audit. What does not survive is the assumption that pre-filtering the

space is a free upgrade for it: three of the four reductions significantly hurt regularised evolution on `cifar10-valid`, it gains significantly in only one of its twelve cells, and half of all significant full-budget gains belong to random search. On tabular benchmarks the cost of statistical adequacy is trivial, as 200 seeds per condition and 16,800 searches complete in 33.5 CPU-hours without a GPU, so seed counts below double digits are no longer defensible for benchmark-based claims in evolutionary NAS.

Limitations. The evidence is scoped to the NAS-Bench-201 topology space and its three image classification datasets; the planned robustness leg on the independent NATS-Bench size space was not run, and the claims are correspondingly scoped rather than universal. The budget is counted in queries, not wall-clock time, which favours neither side at matched size but abstracts away per-architecture cost differences that a deployment would see. The dynamics analysis of Section IV-E covers RE only. The search is single-objective on validation accuracy, and each score-based reduction is instantiated at a single threshold, the published top-50% form; other thresholds would change subspace size and therefore also the matched control, not the protocol. These boundaries should not be read as evidence that reductions fail in open-world spaces; they mark exactly the region where the size-matched control has been applied and where stronger evidence than the full-space comparison is now available.

VI. CONCLUSIONS

At equal query budgets on the NAS-Bench-201 topology space, most of the audited search-space reductions perform no better than size-matched random subspaces at the final budget. Of 48 cells, 27 show no Holm-significant difference, only 12 favour the reduction and 9 the random control, and every effect is at most 0.373 percentage points, below one across-seed standard deviation. The reductions do help early, by 0.71–1.26 pp at 10 queries, and the matched control shows that this help comes from the selection rule rather than from size. It lasts only while the budget is a small fraction of the space, to a median of 50–150 queries for the structure-exploiting searchers, and in several cells it turns into a significant loss. Full-space searches already spend 89–91% of their best trajectories inside the regions the score-based reductions keep, and reductions narrow the end points of evolutionary search without raising their accuracy, so at matched size the reductions largely reshuffle the budget rather than improve its destination. The equal-budget, size-matched protocol, with frozen hashed memberships, charged proposals and paired seed-level statistics, is released with code and subspace definitions for exact reproduction, and is proposed as the standard control for future evaluations of search-space reduction in evolutionary and general NAS. A natural next step, derived from the scope limitation above, is to run the identical protocol on an independent search-space family, on large spaces where practical budgets fall into the regime in which the rules helped here, and on multi-objective formu-

lations where subspace size interacts with the shape of the Pareto front.

APPENDIX

This appendix belongs to the extended version of the paper and collects the displays that the eight-page proceedings version omits for space: the absolute anytime curves of every space (Fig. 4), the paired advantage of every algorithm–reduction pair at every checkpoint (Table VI), the per-cell effects of Table III as a heatmap (Fig. 5) and the descriptive critical-difference diagram of the Friedman–Nemenyi analysis (Fig. 6).

ACKNOWLEDGMENT

The authors used an AI assistant (Claude, Anthropic) for help with drafting and language editing of the text and with code for the statistical analysis and the figures. The authors designed the study, ran all experiments, checked every reported number against the released results, and take full responsibility for the content of the paper.

REFERENCES

- [1] T. Elsken, J. H. Metzen, and F. Hutter, “Neural architecture search: A survey,” *Journal of Machine Learning Research*, vol. 20, no. 55, pp. 1–21, 2019.
- [2] C. White, M. Safari, R. Sukthankar, B. Ru, T. Elsken, A. Zela, D. Dey, and F. Hutter, “Neural architecture search: Insights from 1000 papers,” *arXiv preprint arXiv:2301.08727*, 2023.
- [3] Y. Hu, Y. Liang, Z. Guo, R. Wan, X. Zhang, Y. Wei, Q. Gu, and J. Sun, “Angle-based search space shrinking for neural architecture search,” in *European Conference on Computer Vision (ECCV)*, 2020.
- [4] Y. Ci, C. Lin, M. Sun, B. Chen, H. Zhang, and W. Ouyang, “Evolving search space for neural architecture search,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [5] I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, “Designing network design spaces,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [6] Y. Guo, Y. Chen, Y. Zheng, P. Zhao, J. Chen, J. Huang, and M. Tan, “Breaking the curse of space explosion: Towards efficient NAS with curriculum search,” in *International Conference on Machine Learning (ICML)*, 2020.
- [7] J. Mellor, J. Turner, A. Storkey, and E. J. Crowley, “Neural architecture search without training,” in *International Conference on Machine Learning (ICML)*, 2021, pp. 7588–7598.
- [8] M. S. Abdelfattah, A. Mehrotra, Ł. Dudziak, and N. D. Lane, “Zero-cost proxies for lightweight NAS,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [9] L. Li and A. Talwalkar, “Random search and reproducibility for neural architecture search,” in *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2019.
- [10] A. Yang, P. M. Esperança, and F. M. Carlucci, “NAS evaluation is frustratingly hard,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [11] K. Yu, C. Sciuto, M. Jaggi, C. Musat, and M. Salzmann, “Evaluating the search phase of neural architecture search,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [12] M. Lindauer and F. Hutter, “Best practices for scientific research on neural architecture search,” *Journal of Machine Learning Research*, vol. 21, no. 243, pp. 1–18, 2020.
- [13] X. Dong and Y. Yang, “NAS-Bench-201: Extending the scope of reproducible neural architecture search,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [14] X. Dong, L. Liu, K. Musial, and B. Gabrys, “NATS-Bench: Benchmarking NAS algorithms for architecture topology and size,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3634–3646, 2022.
- [15] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, “Regularized evolution for image classifier architecture search,” in *AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 4780–4789.
- [16] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, pp. 229–256, 1992.
- [17] T. den Ottelander, A. Dushatskiy, M. Virgolin, and P. A. N. Bosman, “Local search is a remarkably strong baseline for neural architecture search,” in *Evolutionary Multi-Criterion Optimization (EMO), LNCS 12654*, 2021, pp. 465–479.
- [18] L. Wang, S. Xie, T. Li, R. Fonseca, and Y. Tian, “Sample-efficient neural architecture search by learning actions for Monte Carlo tree search,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5503–5515, 2022.
- [19] H. Tanaka, D. Kunin, D. L. K. Yamins, and S. Ganguli, “Pruning neural networks without any data by iteratively conserving synaptic flow,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [20] C. White, A. Zela, B. Ru, Y. Liu, and F. Hutter, “How powerful are performance predictors in neural architecture search?” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [21] X. Ning, C. Tang, W. Li, Z. Zhou, S. Liang, H. Yang, and Y. Wang, “Evaluating efficient performance estimators of neural architectures,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.
- [22] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter, “NAS-Bench-101: Towards reproducible neural architecture search,” in *International Conference on Machine Learning (ICML)*, 2019, pp. 7105–7114.
- [23] Y. Mehta, C. White, A. Zela, A. Krishnakumar, G. Zabergja, S. Moradian, M. Safari, K. Yu, and F. Hutter, “NAS-Bench-Suite: NAS evaluation is (now) surprisingly easy,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [24] A. Krishnakumar, C. White, A. Zela, R. Tu, M. Safari, and F. Hutter, “NAS-Bench-Suite-Zero: Accelerating research on zero cost proxies,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022.
- [25] E. Real, S. Moore, A. Selle, S. Saxena, Y. L. Suematsu, J. Tan, Q. V. Le, and A. Kurakin, “Large-scale evolution of image classifiers,” in *International Conference on Machine Learning (ICML)*, 2017, pp. 2902–2911.
- [26] Y. Liu, Y. Sun, B. Xue, M. Zhang, G. G. Yen, and K. C. Tan, “A survey on evolutionary neural architecture search,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 2, pp. 550–570, 2023.
- [27] A. Telikani, A. Tahmassebi, W. Banzhaf, and A. H. Gandomi, “Evolutionary machine learning: A survey,” *ACM Computing Surveys*, vol. 54, no. 8, pp. 1–35, 2021.
- [28] I. Hadjadji, N. E. Karabadi, H. Seridi, N. Manaa, M. Elati, and W. Dhifli, “Optimizing MLP network structure for classification problems using PSO and dominating vertex set,” in *IEEE International Conference on Data Mining Workshops (ICDMW)*, 2024, pp. 529–534.
- [29] N. Hansen, A. Auger, R. Ros, O. Mersmann, T. Tušar, and D. Brockhoff, “COCO: A platform for comparing continuous optimizers in a black-box setting,” *Optimization Methods and Software*, vol. 36, no. 1, pp. 114–144, 2021.
- [30] Y. Jin and J. Branke, “Evolutionary optimization in uncertain environments—a survey,” *IEEE Transactions on Evolutionary Computation*, vol. 9, no. 3, pp. 303–317, 2005.
- [31] J. Derrac, S. García, D. Molina, and F. Herrera, “A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms,” *Swarm and Evolutionary Computation*, vol. 1, no. 1, pp. 3–18, 2011.
- [32] P. Chrabaszcz, I. Loshchilov, and F. Hutter, “A downsampled variant of ImageNet as an alternative to the CIFAR datasets,” *arXiv preprint arXiv:1707.08819*, 2017.
- [33] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [34] J. W. Pratt, “Remarks on zeros and ties in the Wilcoxon signed rank procedures,” *Journal of the American Statistical Association*, vol. 54, no. 287, pp. 655–667, 1959.
- [35] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

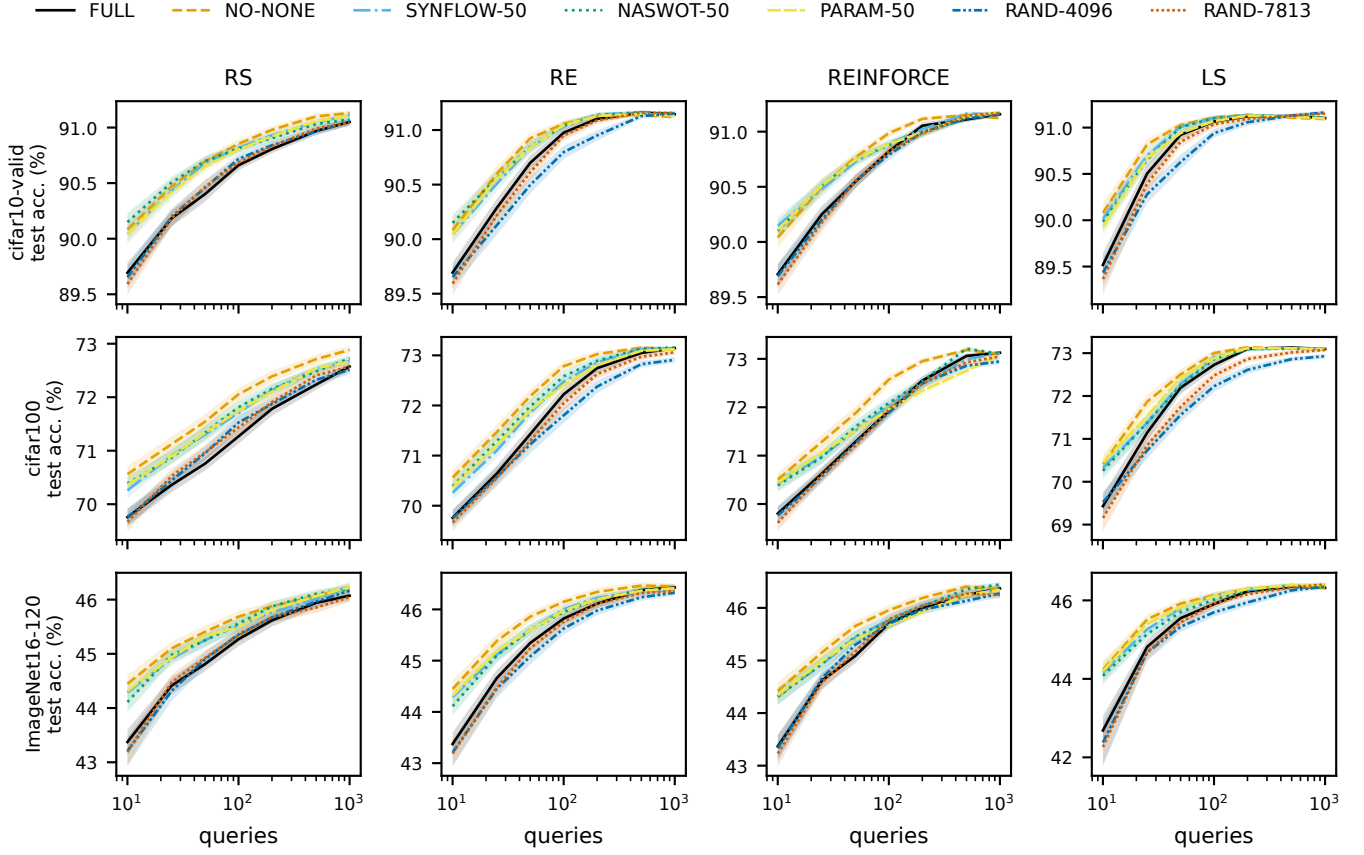


Fig. 4. Anytime mean test accuracy with bootstrap 95% confidence bands across budgets 10–1,000 (rows: datasets; columns: search algorithms). Early advantages of the reductions over their size-matched random controls decay with budget and, for local search under the score-based reductions, reverse by the final budget (Table IV).

TABLE VI

MEAN PAIRED DIFFERENCE IN ANYTIME TEST ACCURACY (PERCENTAGE POINTS, REDUCTION MINUS SIZE-MATCHED RANDOM CONTROL) AT EACH BUDGET CHECKPOINT, AVERAGED OVER THE THREE DATASETS WITH PAIRING BY SEED. EVERY PAIR STARTS AHEAD AT $B=10$; RS, RE AND LS DECAY MONOTONICALLY, REINFORCE DIPS BY MID-BUDGET AND ONLY PARTLY RECOVERS. BOTTOM ROWS AS IN TABLE IV.

Algorithm	Reduction	$B=10$	25	50	100	200	500	1,000
RS	NO-NONE	+0.83	+0.58	+0.44	+0.33	+0.29	+0.21	+0.17
	SYNFLOW-50	+0.71	+0.36	+0.29	+0.20	+0.14	+0.12	+0.12
	NASWOT-50	+0.73	+0.39	+0.31	+0.24	+0.17	+0.12	+0.11
	PARAM-50	+0.76	+0.36	+0.32	+0.20	+0.14	+0.14	+0.11
RE	NO-NONE	+0.83	+0.77	+0.72	+0.59	+0.40	+0.19	+0.10
	SYNFLOW-50	+0.71	+0.49	+0.39	+0.25	+0.14	+0.06	+0.03
	NASWOT-50	+0.73	+0.57	+0.43	+0.27	+0.14	+0.08	+0.05
	PARAM-50	+0.76	+0.57	+0.40	+0.23	+0.11	+0.05	+0.03
REINFORCE	NO-NONE	+0.73	+0.50	+0.39	+0.38	+0.27	+0.21	+0.07
	SYNFLOW-50	+0.81	+0.34	+0.25	+0.04	+0.01	+0.02	+0.05
	NASWOT-50	+0.78	+0.35	+0.29	+0.05	+0.01	+0.12	+0.09
	PARAM-50	+0.81	+0.40	+0.25	−0.01	−0.08	−0.07	+0.05
LS	NO-NONE	+1.13	+0.84	+0.65	+0.46	+0.32	+0.11	+0.02
	SYNFLOW-50	+1.24	+0.47	+0.35	+0.23	+0.13	+0.05	−0.02
	NASWOT-50	+1.18	+0.42	+0.31	+0.18	+0.11	+0.04	−0.04
	PARAM-50	+1.26	+0.56	+0.39	+0.21	+0.14	+0.05	−0.02
Cells significantly better		48	48	47	35	25	13	12
Cells significantly worse		0	0	0	0	0	2	9

[36] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.

[37] J. L. Hodges and E. L. Lehmann, “Estimates of location based on rank tests,” *The Annals of Mathematical Statistics*, vol. 34, no. 2, pp. 598–

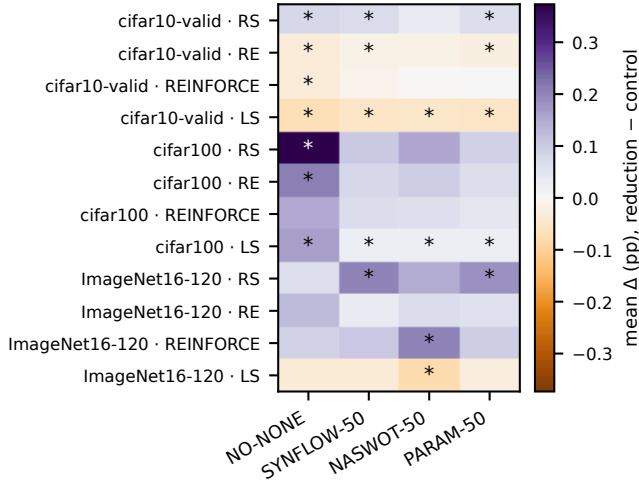


Fig. 5. Mean paired difference (reduction minus matched random control, percentage points) per dataset–algorithm–reduction cell, with Holm significance stars from Table III. Sign flips along the algorithm axis: reductions that help random search can hurt local search on the same dataset.

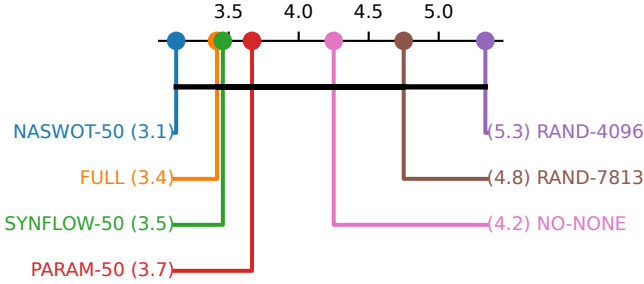


Fig. 6. Critical-difference diagram over the seven spaces (Friedman–Nemenyi, blocks are the 12 dataset–algorithm cells). The Friedman omnibus does not reject ($p = 0.121$), so the diagram is descriptive: the random controls rank last, yet no space is separated from FULL at $\alpha = 0.05$.

611, 1963.

- [38] M. Friedman, “The use of ranks to avoid the assumption of normality implicit in the analysis of variance,” *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.
- [39] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.